

Chapter 8

In Class Examples

Slides 1-7

1. Let's start off by doing some simple examples. Create an anonymous function that will square the input.

```
test=@(x) x.^2
```

```
test(2)
pause
x=0:4;
test(x)
pause
y=3:2:7;
test(y)
```

Notice that the input variable used doesn't have to match what's in the function.

2. Download grades.txt from the website. This file contains homework and exam grades for 10 students. Use an anonymous function to calculate the final grades for each of the students.

```
load grades.txt
final_grade=@(hw,t1,t2,f) hw*.10+t1*.25+t2*.25+f*.40;
final_grade(grades(:,1),grades(:,2),grades(:,3),grades(:,4))
```

3. Use anonymous functions to plot three equations. Use x values ranging from 1 to 25 in 0.5 increments.

```
func1=@(x) 1./x;
func2=@(x) x;
func3=@(x) x.^2

x=1:.5:25
subplot(1,3,1)
plot(x,func1(x))
axis tight
title('1/x')
subplot(1,3,2)
plot(x,func2(x))
axis tight
title('x')
subplot(1,3,3)
plot(x,func3(x))
axis tight
title('x^2')
```

Slides 8-13

4. Find the solution to $e^{-x} \sin\left(\frac{x}{2}\right) = 0.75$. To do this we will use the built in function root finding method in MATLAB: `fzero`. This built in function essentially does the Newton-Raphson Method. Check the accuracy of the approximation by using the value in the equation. Syntax for the function: `fzero(equation, initial approximation)`

```
f = @(x) exp(-x)*sin(x/2)-0.75
x_approx=fzero(f,2.5)
f(x_approx)
```

5. Use Newton's method to find, to 8 significant figures, the positive roots of the following equation. Use a tolerance of 10^{-8} .

$$10e^{\sin x} = x^2 - 5x + 4.$$

This time we will write the function function that will use Newton's method to find the root. Below is the algorithm for the method.

Inputs: function $f(x)=0$

p_0 – initial approximation of root

N_0 – max #of iterations

TOL- tolerance

Output: p – root approximation

Algorithm:

Set $i=1$

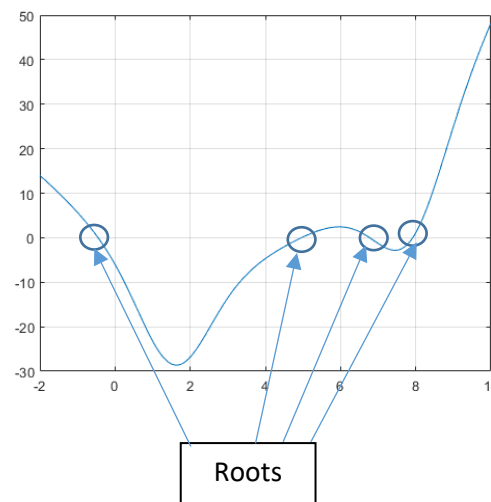
While $i \leq N_0$ and $\text{abs}(p-p_0) > \text{TOL}$

$p = p_0 - f(p_0)/f'(p_0)$

$i = i + 1$

$p_0 = p$

OUTPUT (Failure Message) if no solution



Script file:

```
format long

po=-1;
tol=10^-8;
fp=@(po) po.^2-5*po+4-10*exp(sin(po));
estimate=newton(fp,@deriv_func,po,tol)
x=[-2:.05:10];
plot(x,fp(x))
grid on
```

Write a user-defined function to find the derivative of $x^2 - 5x + 4 - 10e^{\sin x}$:

```
function [dfp] = deriv_func( po )
dfp=2*po-5-10*cos(po)*exp(sin(po));
end
```

continued on next page...

Create the function `newton.m`:

```
function [p] = newton( fp,dfp,po,tol )
n=100;
error=100;
i=0;

while (i<n & error>tol)
    p=po-(fp(po)/dfp(po))
    error=abs(p-po);
    i=i+1;
    po=p;
end
end
```

Slide 14-16

6. Create anonymous functions to convert temperatures. Save these functions in a .mat file called `tempconv.mat`.

```
clear all
KtoC=@(T_Kelvin) T_Kelvin-273.15;
RtoF=@(T_Rankine) T_Rankine-459.67;
CtoF=@(T_Celsius) 1.8*T_Celsius+32;
KtoR=@(T_Kelvin) 1.8*T_Kelvin;
save tempconv
clear
load tempconv
```

7. Use `fplot` to create a plot of $e^{-x} \sin\left(\frac{x}{2}\right) = 0.75$. Use x values from 0 to 8. Plot with a red line.

```
f = @(x)exp(-x)*sin(x/2)-0.75;
fplot(f,[0,8], 'r')
```

Slides 17-30

8. Plot a first-order response function $c(t)$. Allow the user to specify the plot color if desired. If no color is specified then the default plot color (blue) is used. Create a function to create the plot and allow variable inputs.

$$c(t) = 2 - 2e^{-2t}$$

Script file

```
color1=input('Enter the plot color if desired:\n','s')
t=0:50;
c = 2-2*exp(-2*t);
if isempty(color1)
    plotc(t,c)
    xlabel('Time')
    ylabel('c(t)')
else
    plotc(t,c,color1)
    xlabel('Time')
    ylabel('c(t)')
end
```

plotc.m

```
function plotc( t,c,varargin)

if nargin==3
    plot(t,c,varargin{1})
else
    plot(t,c)
end
end
```

9. A function “convyards” is written to help runners calculate how far they ran. The function receives as input the number of yards. It calculates and returns the equivalent number of meters, and, if (and only if) two output arguments are expected, it also returns the equivalent number of miles. The relevant conversion factors are:

1 meter = 1.0936133 yards

1 mile = 1760 yards

Script file

```
yards=input('Enter the number of yards ran:\n');
choice=menu('Choose the desired conversion','meters','meters and miles');
if choice==1
    [meters]=convyards(yards);
    fprintf('Distance ran is %.2f meters\n',meters)
elseif choice==2
    [meters,miles]=convyards(yards);
    fprintf('Distance ran is %.2f meters or %.2f miles\n',meters,miles)
end
```

convyards.m

```
function [meters, varargin] = convyards(yards)
meters = yards / 1.0936133;
if nargin == 2
    varargin{1} = yards / 1760;
end
end
```

10. Write a function that will receive a variable number of input arguments: the length and width of a rectangle, and possibly also the height of a box that has this rectangle as its base. The function should return the rectangle area if just the length and width are passed, or also the volume if the height is also passed.

Script file

```
length=input('Enter the length of the box:\n');
width=input('Enter the width of the box:\n');
height=input('If volume is needed, enter the height. Otherwise hit the enter...
button to continue.\n')

if isempty(height)
    area=boxcalcs(length,width);
    fprintf('The area of the rectangle is %.2f.\n',area)
else
    [area,volume]=boxcalcs(length,width,height);
    fprintf('The area of the rectangle is %.2f. Volume is %.2f.\n',area,volume)
end
```

boxcalcs.m

```
function [area, varargout] = boxcalcs(len, wid, varargin)
% Calculates a rectangle area and possibly also a
% height of a box and returns the area and possibly volume
% Format of call: boxcalcs(length, width) or
%   boxcalcs(length, width, height)
% Returns area of box and if height is passed, volume

% always return the area
area = len * wid;

% if the height was passed, return the box volume
if nargin==2 %if nargin == 3   could also be used
    varargout{1} = area * varargin{1};
end
end
```